

Week 6 — Spark Architecture, Performance & Optimization

Answers to Assignment Questions Q1-Q15

Q1: Roles of Driver, Cluster Manager, and Executor

Component	Role
Driver	The process that runs the <code>main()/SparkSession</code> . It converts user code into a logical plan, builds the DAG of stages, schedules tasks, and coordinates the overall application. It also collects results returned by actions like <code>.collect()</code> .
Cluster Manager	Allocates cluster resources (CPU, memory) to the Spark application and launches Executor processes on worker nodes. Options: Standalone, YARN, Kubernetes , or <code>local[*]</code> for single-machine testing.
Executor	A JVM process on a worker node that actually runs tasks assigned by the Driver, stores cached/persisted data, and writes shuffle output. Each application gets its own set of Executors for the duration of the job.

Flow: Driver → requests resources from Cluster Manager → Cluster Manager launches Executors → Driver sends tasks to Executors → Executors execute and report results/status back to the Driver.

Q2: Lazy Evaluation and Chain-Processing Performance

Spark does **not** execute transformations (`filter`, `select`, `withColumn`, `join`, etc.) as soon as they're called. Instead, it builds up a **logical plan** describing what should happen, and only executes when an **action** (`.count()`, `.show()`, `.write()`, `.collect()`) is triggered.

This improves performance in chain-processing because:

1. **Whole-pipeline optimization** — Spark's Catalyst optimizer sees the *entire* chain at once and can reorder, merge, or eliminate operations (e.g., push a filter down before a join, combine multiple map-like steps into a single pass).

2. **Predicate/column pushdown** — filters and required columns can be pushed all the way down to the data source scan, so unnecessary data is never read from disk in the first place.
3. **No wasted intermediate materialization** — without laziness, each transformation would need to fully compute and store its output before the next step; with laziness, Spark can pipeline many narrow transformations into a single stage.
4. **Avoids unnecessary computation** — if a later step in the chain filters out most data, Spark's optimizer can apply that filtering earlier, cutting work rather than processing rows that would be discarded anyway.

The net effect: a long chain of `.filter().select().withColumn().groupBy()` compiles down to one efficient, optimized physical execution plan instead of many separate, naively-executed steps.

Q3: Read CSV with Header and inferSchema

```
df = spark.read.csv("data/source.csv", header=True, inferSchema=True)
```

Or the equivalent option-based syntax:

```
df = (
    spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("data/source.csv")
)
```

Note: `inferSchema=True` requires an extra pass over the data to detect types, which adds overhead and can misfire on inconsistent formats (e.g., mixed date formats) — an explicit `StructType` schema is safer for production pipelines.

Q4: CSV vs. Parquet — Row-Based vs. Columnar Storage

Aspect	CSV (row-based)	Parquet (columnar)
Layout	Stores complete rows sequentially — all columns of row 1, then all columns of row 2, etc.	Stores each column's values together, contiguously.
Compression	Poor — mixed data types per row limit compression ratio.	Excellent — same-type values compress much better (e.g., Snappy/GZIP), often 3-10x smaller files.
Column pruning	Must read every column of every row even if only 2 of 20 columns are needed.	Can read only the requested columns directly off disk, skipping the rest entirely.

Aspect	CSV (row-based)	Parquet (columnar)
Predicate pushdown / stats	No stored statistics — every row must be parsed to check a filter.	Stores min/max statistics per row-group , letting Spark skip entire row-groups without reading them.
Schema	No embedded schema — types must be inferred or declared separately.	Schema is embedded in the file itself.

Why it matters for performance: with Parquet, a query that only needs 2 columns and filters on one of them can skip both irrelevant columns *and* irrelevant row-groups — dramatically reducing disk I/O and memory usage compared to CSV, which forces a full row-by-row, column-by-column scan every time. This gap grows enormously at scale (millions/billions of rows).

Q5: Select product_id, price Where category == 'Electronics'

```
from pyspark.sql import functions as F
result = df.filter(F.col("category") == "Electronics").select("product_id", "price")
```

Q6: Rename old_name → new_name and Cast price to Double

```
from pyspark.sql.types import DoubleType
df_revised = (
    df.withColumnRenamed("old_name", "new_name")
      .withColumn("price", F.col("price").cast(DoubleType()))
)
```

Because DataFrames are **immutable**, both operations return new DataFrames — the original df is never modified in place, which is why the result must be captured (here, in df_revised).

Q7: Lineage Graph (DAG) and Fault Tolerance

Spark DataFrames/RDDs are built from a **lineage graph** — a record of the sequence of transformations (and the source data) used to derive each dataset, rather than the dataset's actual contents being duplicated for safety.

How this provides fault tolerance:

1. If a worker node/Executor fails (e.g., crashes, is preempted, or loses network connectivity) partway through a job, any **partitions** that were being processed or cached on that node are lost.
2. Because Spark knows the **lineage** — the exact chain of transformations that produced those partitions from the original source data — it can simply **recompute only the lost partitions** by re-running that portion of the DAG on a different, healthy Executor.
3. This avoids needing to replicate all data multiple times (as some systems do) — recovery is based on **recomputation from lineage**, not from data replication.
4. The **DAG scheduler** re-issues tasks for the missing partitions as a normal part of scheduling; from the application's perspective, this is largely transparent (aside from added latency for the retry).

This is a key architectural difference from traditional distributed systems that rely purely on replication for fault tolerance — Spark's lineage-based approach is more memory/storage efficient, at the cost of some recomputation time on failure.

Q8: Filter df_orders — status == 'Completed' AND amount > 1000

```
result = df_orders.filter(
    (F.col("status") == "Completed") & (F.col("amount") > 1000)
)
```

Q9: Predicate Pushdown in Parquet

Predicate pushdown means Spark pushes a filter condition (a “predicate,” e.g. WHERE department = 'Engineering') down to the **data source/file-scan level**, rather than reading all the data into memory first and filtering afterward.

How it works with Parquet specifically:

- Parquet files are divided into **row-groups**, each storing **min/max statistics** for every column.
- When Spark sees a filter like price > 500, it checks each row-group's stored min/max for price. If a row-group's max is 450, Spark knows **no row in that entire row-group can match** and skips reading it from disk entirely — no decompression, no deserialization, no memory allocation for those rows.
- Only row-groups that *could* contain matching rows are actually read, decompressed, and scanned row-by-row for the exact match.

Effect on memory: because entire row-groups can be skipped before ever being loaded, predicate pushdown **substantially reduces the amount of data read from disk and materialized in memory** — often by a large margin when filters are selective. This is a key reason Parquet queries can be far faster and lighter on memory than equivalent CSV queries, where every row must be read and parsed regardless of the filter (CSV has no stored statistics to skip on).

Q10: Add Column `final_price = base_price * 1.18`

```
df = df.withColumn("final_price", F.col("base_price") * 1.18)
```

Q11: Transformations vs. Actions

	Transformations	Actions
Definition	Operations that produce a new DataFrame/RDD from an existing one. Lazy — not executed immediately.	Operations that trigger execution of the built-up plan and return a result to the Driver or write output.
Execution	Only build up the logical plan/DAG.	Cause Spark to actually run the DAG across the cluster.
Examples	<code>filter()</code> , <code>select()</code> , <code>withColumn()</code> , <code>groupBy()</code> , <code>join()</code> , <code>map()</code>	<code>count()</code> , <code>show()</code> , <code>collect()</code> , <code>write()</code> , <code>take()</code>

Two examples of each (as requested): - Transformations: `df.filter(F.col("age") > 18)`, `df.withColumn("total", F.col("a") + F.col("b"))` - Actions: `df.count()`, `df.show()`

Q12: Read Parquet → Filter Null `user_id` → Write CSV

```
df = spark.read.parquet("path/to/input")
df_filtered = df.filter(F.col("user_id").isNull())
df_filtered.write.mode("overwrite").option("header", True).csv("path/to/output")
```

Q13: Client Mode vs. Cluster Mode

	Client Mode	Cluster Mode
Driver location	Runs on the machine that submits the job (e.g., your laptop, an edge node, a notebook server) — outside the cluster's managed nodes.	Runs inside the cluster , on one of the worker nodes, managed by the Cluster Manager (YARN/Kubernetes/Standalone).

	Client Mode	Cluster Mode
Typical use case	Interactive work: spark-shell, notebooks, debugging — you want to see Driver logs/output directly in your terminal.	Production/unattended batch jobs submitted via spark-submit --deploy-mode cluster — the submitting machine can disconnect without killing the job.
Reliability	If the submitting machine or its network connection goes down, the Driver (and the whole application) dies with it.	The Driver survives independently of the machine that submitted the job, since it runs inside the cluster.
Network	More network traffic between the external Driver and the in-cluster Executors.	Driver and Executors are co-located in the cluster, reducing cross-network chatter.

Q14: Filter — region == 'North' OR priority == 'High'

```
result = df.filter(
    (F.col("region") == "North") | (F.col("priority") == "High")
)
```

Q15: Why .show(5) Instead of .collect() on a Multi-Terabyte Dataset

- **.collect() pulls the *entire* result set back to the Driver's memory** as a single in-memory collection. On a multi-terabyte dataset, this means attempting to load terabytes of data into one JVM process — something the Driver almost certainly does not have the RAM for, leading to an **OutOfMemoryError** and a crashed application.
- **.show(5) only computes and materializes a small sample** (the first 5 rows by default) and prints it directly, without ever bringing the full dataset to the Driver. Spark can often short-circuit execution once enough rows are produced, avoiding a full-dataset computation just to preview it.
- **Distributed data should generally stay distributed.** The Driver is a single process with a fixed memory budget; the whole point of Spark's Executor model is to keep large data spread across many machines rather than concentrated in one place. .collect() defeats this by centralizing everything.
- **Safer exploration workflow:** use .show(), .take(n), .limit(n).toPandas() (for genuinely small results), or write full results to disk (.write.parquet(...)) and inspect them with sampling tools — reserving .collect() only for results you are certain are small (e.g., aggregated summaries, not raw multi-terabyte tables).

Summary Insight

Across these questions, three themes recur: (1) Spark's **architecture** (Driver/Cluster Manager/Executor) separates planning from execution across a cluster; (2) **laziness + the DAG/lineage graph** is what enables both performance optimization (pushdown, plan rewriting) *and* fault tolerance (recomputation instead of replication); and (3) **choosing the right file format and API calls** (Parquet over CSV, `.show()` over `.collect()`) has an outsized impact on real-world performance and stability at scale.